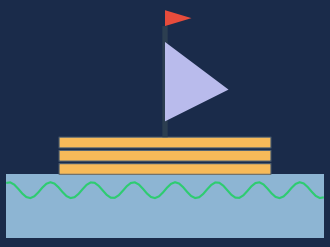
One Piece: Raft Survivor

Phase 2 — Technical Design Document

Member	Name	Task Ownership (Vertical Slices)
Member 1	*(Team Member 1)*	Task 1: Foundations & The Ocean
Member 2	*Mohamed*	Task 2: Survival & Crafting
Member 3	*Ahmed Gamal*	Task 3: Ocean Predators (Sharks & Marines)
Member 4	*Yousef*	Task 4: Epic Encounters (Kraken & Penguin)

Computer Graphics · Due: 27 April 2026



0 Contents

1 Project Overview & Phase Structure 2

2 Team Breakdown & Parallel Work Strategy 2

3 Game Flow & Wave State Machine 2

4 Phase A — Lighting Architecture 3
- 4.1 New Files . . . 3
- 4.2 Modified Files . . . 3
- 4.3 LightComponent . . . 3
- 4.4 LitMaterial . . . 4
- 4.5 Phong GLSL Shader Outline . . . 4

5 Phase B — Core Game Systems 4
- 5.1 Raft System . . . 5
- 5.2 Player Controller . . . 5
- 5.3 Health & Resource System . . . 5

6 Phase C — Enemies & Combat 5
- 6.1 Enemy Types Overview . . . 5
- 6.2 Enemy Systems . . . 6
- 6.3 Devil Fruit Powers . . . 6

7 Phase D — Ocean Water Shader 6
- 7.1 New Files . . . 6
- 7.2 Vertex Displacement . . . 6
- 7.3 Fragment — Fresnel + Foam . . . 7
- 7.4 Postprocess Effect (Phase 2 Requirement) . . . 7

8 Phase E — Boss, HUD & Scene Management 7
- 8.1 Gembilyas Boss System . . . 8
- 8.2 2D HUD Renderer . . . 8
- 8.3 Scene JSON Files . . . 8

9 3D Model Assets 8

10 Complete New File Registry 9

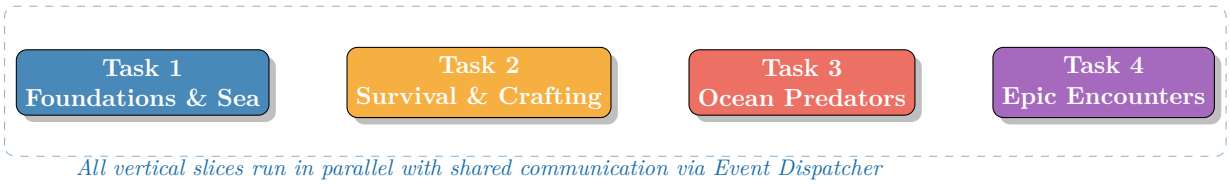
11 PDF Report Requirements 9

12 Task Division — 4-Member Team 11

1 Project Overview

This document outlines the complete technical plan for **Phase 2** of the One Piece: Raft Survivor game, built on a custom C++ OpenGL Engine using an Entity Component System (ECS). Phase 2 shifts toward a full implementation of the game loop: interacting with the sea environment, fighting oceanic beasts and marines, progressing through waves of enemies, and finally meeting a giant penguin to win the "One Piece".

To ensure all members get hands-on experience with both **Graphics/Shaders (OpenGL)** and **Core Logic (C++)**, work is divided into **Vertical Slices**. This means each member owns a specific feature from the mathematical shader rendering up to the gameplay loops.



2 Game Flow & Wave State Machine

The game flow is separated into chronological sequences that challenge the player's resource management and combat skills.

3 Task 1 — Foundations & The Ocean

3.1 Overview

Build the core environment, moving sea elements, and basic player controls.

- **Graphics (OpenGL):**

  - **Sea Shader:** Implement Gerstner Waves math in `water.vert` to create rolling, peaked waves so that the water is not just a flat plane.
  - **Skybox/Lighting:** Setup a cubemap for the clear sky and sun as the main light source. Integrate basic Phong or PBR lighting so the scene casts proper shadows/lighting on the raft.

- **Logic (C++):**
  - **Input & Movement:** Map WASD, Space for jumps, and Motion Camera.
  - **Buoyancy Math:** Replicate the Gerstner wave calculations in C++ to compute the exact (x, z) height. Ensure the boat and player "bob" on the waves rather than clipping through them.

4 Task 2 — Survival & Crafting (The Raft Core)

4.1 Overview

Manage the player's core gameplay loop of staying alive, collecting resources, and UI overlays.

- **Graphics (OpenGL):**
  - **HUD Rendering:** Draw 2D overlays using Orthographic projection. Position the Health bar (bottom left), Devil Fruit power bar (above health), and Inventory selection (bottom).
  - **Mesh/Textures Strategy:** Load missing chunk models. When a part of the boat is destroyed, swap the visible plank mesh so it visually breaks down.

- **Logic (C++):**
  - **Inventory System:** Store items (Hammer, Spear, Net/Fishing rod). Map keys 1, 2, 3, 4 to equip tools respectively. Handle fishing states.
  - **Health/Repair System:** Logic for eating fish (restores Player HP) and fixing the boat using wood (restores Ship HP and re-enables missing mesh chunks).

5 Task 3 — Ocean Predators (Sharks & Marines)

5.1 Overview

Spawn enemies, calculate AI movements, and handle incoming damage and ship projectiles.

- **Graphics (OpenGL):**
  - **Model Management:** Use Assimp to load Shark and Marine Ship templates.
  - **Damage Visuals:** Shader effect for hit indicators (e.g., flash red when players strike a shark). Render cannonballs and bullets flying through the air.

- **Logic (C++):**
  - **Shark AI:** 3-5 Sharks that circle the boat, approach into melee range, and deplete ship HP. Calculate player spear intersections using Raycasting. Disburse a Devil Fruit drop on full wave completion.
  - **Marine AI:** Implement 2 ship sizes. Manage ships firing cannons and bullets (projectiles) toward the player's location. Keep an internal timer/counter for the invasion retreat condition.

6 Task 4 — Epic Encounters (Kraken & Penguin)

6.1 Overview

Develop complex boss attack phases, animations, large UI elements, and the win condition sequence.

- **Graphics (OpenGL):**
  - **Boss Animations:** Drive the Kraken tentacles using hierarchical transforms or Inverse Kinematics (IK). Visually telegraph when an attack is about to swing.
  - **Boss UI:** Implement the giant boss health bar at the top of the screen.
- **Logic (C++):**
  - **Kraken State Machine:** Wait → Windup (Vulnerable) → Swing (Damages Player and Ship). Drop Giant Feather + Fruit shards on death.
  - **Penguin & Win Condition:** Create the giant sad penguin interaction. Detect if the player currently has the Giant Feather equipped to "tickle" it, triggering the happiness state and final Win screen sequence.

7 Event-Driven Architecture (Implementation Tip)

To ensure these tasks operate seamlessly without tightly coupling the codebase (i.e. to avoid a giant "if-else" mess), Phase 2 will heavily utilize an **Event Dispatcher Pattern**.

Listing 1: Event Dispatcher Example

```cpp
// 1. Define typical game events
enum class GameEvent { PLAYER_HIT, SHARK_DEATH, BOAT_DAMAGED };

// 2. Systems can "fire" events decoupled from the receivers
void SharkSystem::Update() {
    if (sharkBitesBoat) {
        EventDispatcher::Trigger(GameEvent::BOAT_DAMAGED,
            damageAmount);
    }
}

// 3. Other systems listen and react asynchronously
```

```
void HudSystem::OnNotify(GameEvent event, int value) {
    if (event == GameEvent::BOAT_DAMAGED) {
        FlashScreenRed();
        LowerShipHealth(value);
    }
}
```

8 Assets & Libraries Guide

3D Models (.obj, .fbx):

- **Sketchfab** & **Poly.pizza:** High quality and low-poly assets with Creative Commons licensing. Perfect for generic "One Piece" looking characters, rafts, and tools.
- **Mixamo (by Adobe):** For humanoid player models. You can upload a standard T-pose model and download walking/running/attack animations.
- **The Models Resource:** Ripped assets from old games. Good for placeholders.

Textures & Skies:

- **Poly Haven:** Best for high-quality PBR textures (Wood planks) and HDRIs for Skyboxes.

C++ Libraries to Utilize:

- **Assimp:** Crucial for loading '.obj' or '.fbx' models into the OpenGL runtime.
- **GLM:** Essential for executing Matrix algorithms, cross products, and camera setup.
- **STB_Image:** Single-header library ideal for texture loading.

9 PDF Report Requirements

Phase 2 PDF Report due: 27 April 2026

1. Team member names and IDs
2. Contribution breakdown per member (map each feature to a name based on the Vertical Slice owned)
3. At least **3 screenshots** from the running game showcasing Bosses, shaders, and UI.

End of Phase 2 Technical Design Document